

MetroMitra

Architecture Document

v1.0 Baseline — Build Cycle 1

MetroMitra Engineering



Companion to docs/PRD.md and docs/TechRD.md

Internal architecture review document

Abstract

MetroMitra is a hyperlocal community web application anchored to Indian metro stations. This document describes the architecture of the v1.0 build: a server-rendered Next.js application backed by Prisma and SQLite, authenticated with NextAuth credentials and JWT sessions, deployed to Vercel from GitHub via a single GitHub Actions pipeline. The intent is to record, for an internal architecture review, the components we chose, the components we deliberately did not choose, the data model, the trust and safety primitives, and the limits we are aware of. Where a more sophisticated solution exists, we note it as a roadmap item rather than retrofit it. The document is written to be read end-to-end, not skimmed for bullet points.

Contents

1	Introduction	1
2	System overview	1
3	Technology stack and rationale	1
3.1	Core stack	2
3.2	What we deliberately did not add	3
3.3	Dependency policy	3
4	Data model	3
4.1	Trust score	5
4.2	Indices and query patterns	5
5	Authentication and session design	5
5.1	Provider and hashing	5
5.2	Session shape	6
5.3	Protected routes	6
5.4	Trust score as session data	6
6	API design	6
6.1	Conventions	6
6.2	Validation contract	7
6.3	Request flow	8
6.4	Errors and the envelope	8
7	Frontend architecture	8
7.1	Route groups	8
7.2	Rendering strategy	9

7.3	State management	9
7.4	Component contract	9
7.5	Motion level	9
8	Deployment and CI/CD	9
8.1	Package manager and lockfile	9
8.2	Topology	9
8.3	CI/CD pipeline	10
8.4	The database limitation	10
9	Security and privacy	11
10	Accessibility and performance	11
11	Limitations and future work	11
12	Conclusion	12
	References	12

1 Introduction

MetroMitra exists because of a specific, observable pattern of behaviour in Indian cities. Several million people pass through the same metro stations every working day, and yet the commute is a solitary experience. They share the platform, the coach, and the exit staircase with hundreds of strangers and have no shared place to find a last-mile ride, recover a dropped phone, find a trustworthy companion for an early-morning leg, or talk about the station they share. Existing products touch pieces of this. BlaBlaCar handles intercity carpooling, OLX handles classifieds, WhatsApp groups handle ad-hoc coordination, and the metro corporations' own apps handle ticketing and journey planning. None of them treats the metro station as the primary unit of community.

MetroMitra is built to test that thesis. A user signs up, anchors their profile to a home station and a work or college station, and immediately sees a station-scoped feed, active carpools originating from that station, lost-and-found entries, and a local marketplace. Trust is not assumed; it is constructed from a profile-completion baseline, peer ratings after completed interactions, and a report flow that feeds a manual moderation queue. The product is positioned for commuters in Delhi, Mumbai, Bengaluru, Hyderabad, Chennai, and Kolkata, with a seed directory of 39 stations across those six cities and three demo users for review.

The build cycle covered by this document is v1.0. It ships the minimum feature set that proves the product thesis, follows the project's Ponytail principle of simplest correct implementation, and is explicit about what it does not yet do. The remainder of this document describes the system in enough detail that an engineer joining the team could orient themselves in a single read.

2 System overview

The application is a single Next.js 16 process running on Vercel's Node.js runtime, talking to a relational database through Prisma. There is no separate backend service, no cache layer, no message queue, no websocket service. Each of those was considered and rejected for v1; the rationale is in Section 3.

Figure 1 shows the components. Browser traffic reaches the Next.js process over HTTPS on the same origin. The process hosts three categories of work: React Server Components for public, SEO-sensitive surfaces; Route Handlers under `/api/*` for validated writes and structured reads; and the NextAuth.js v4 integration that handles credentials authentication and JWT session management. All three share a single Prisma client, which talks to a SQLite file in development and to a hosted SQL database in production.

The choice to keep everything inside Next.js is deliberate. The Ponytail ladder asks, for every candidate piece of infrastructure, whether it needs to exist. A separate backend would split type definitions, double the deployment surface, and force an internal network contract for a v1 product that has no traffic to justify one. A cache layer would shadow bugs in the data layer. WebSockets would add operational complexity for a feature set that is satisfied by TanStack Query's polling interval. The simplest correct architecture for v1 is the one in Figure 1.

3 Technology stack and rationale

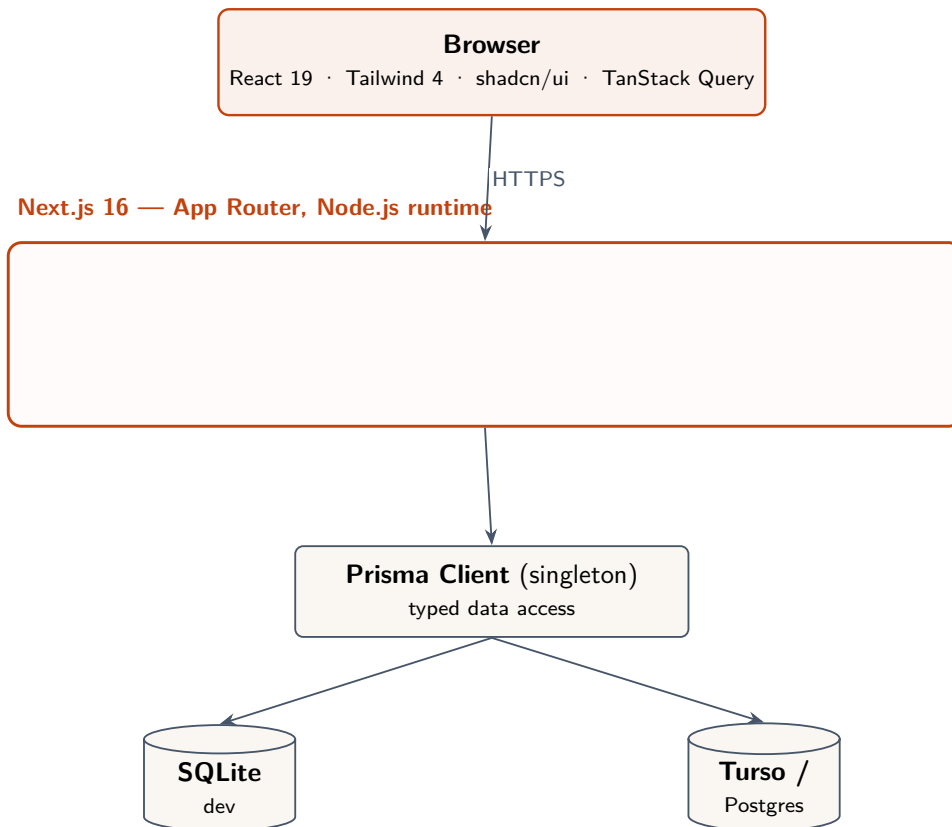


Figure 1: System components. A browser session enters the Next.js process over HTTPS; server components, route handlers, and NextAuth share a single Prisma client, which talks to SQLite locally and to a hosted SQL database in production.

3.1 Core stack

The stack was fixed by the project brief and then validated against the Ponytail ladder. Each component answers a specific concern, and where two components could answer the same concern, we picked the one already installed.

Next.js 16 with the App Router is the framework. It gives us server components for crawlable HTML, route handlers for the API surface, file-based routing, and first-class Vercel deployment. We do not use the edge runtime; route handlers run on the Node.js runtime because Prisma's SQLite driver and NextAuth's credentials provider both expect it.

TypeScript is configured in strict mode. There are no **any** types in application code; trust-boundary inputs are parsed with Zod and the inferred type is propagated through the rest of the call. The TypeScript compiler is part of the CI gate and is treated as a test.

Tailwind CSS 4 with the shadcn/ui component library (New York variant) handles the interface. Tailwind 4's design-token model is a good fit for a brand that needs a small number of roles reused consistently: canvas, ink, primary, line accents, and the standard semantic tokens. shadcn/ui is not a dependency in the npm sense; the component source lives in the repository under `components/ui` and is themed by editing tokens. That means our terracotta primary and metro-line accent colours are applied once, at the token layer, and propagate to every primitive.

Prisma 6 is the ORM. We use it for typed queries, schema-as-documentation, and migration management. The Prisma client is a singleton in `lib/db.ts` to avoid exhausting SQLite connections in development, where Next.js hot-reloading can create many client instances if not guarded.

SQLite is the development database. It is zero-infra: a single file at `db/custom.db`, ignored by git, populated by a seed script that creates the 39 station rows and three demo users. Production is portable to Turso (libSQL) or Postgres by changing the datasource provider and `DATABASE_URL`; no schema changes are required. Section 8 describes the Vercel-specific limitation that motivates that migration.

NextAuth.js v4 with the Credentials provider handles authentication. We use the JWT session strategy, not database sessions, so the deployment stays stateless and Vercel-friendly. Passwords are hashed with Node's built-in `scrypt` rather than `bcrypt`, to avoid native compilation on Vercel's build environment. Section 5 covers the hashing scheme and session shape in detail.

TanStack Query manages server state on the client. All list and detail fetches go through it, with sensible `staleTime` values: 60 seconds for feeds, 30 seconds for carpool and buddy lists, and `stale-while-revalidate` for everything else. Optimistic updates are used sparingly; for v1 most mutations invalidate the relevant query keys and let the refetch settle the UI.

Zustand holds client-side UI state that is not server state: the SOS dialog open flag, the active station filter, transient form values that span components. It is small and has no boilerplate, which is the point.

Framer Motion is used at Level 1 only. Refined interface motion: hover, focus, press, dialog open and close, tab switch, section entry. We do not use it for hero animations, page transitions, or anything that would suggest a marketing site. `prefers-reduced-motion` is respected throughout; motion degrades to opacity-only or none.

3.2 What we deliberately did not add

The Ponytail ladder asks, for every candidate addition, whether it needs to exist. A separate backend service was the first candidate rejected. Next.js route handlers satisfy every API requirement and keep the deployment as one unit; a second service would split type definitions, double the deploy surface, and force an internal network contract for a v1 product with no traffic to justify one. A cache layer was the next. TanStack Query handles client caching, Prisma's connection handling is sufficient at v1 scale, and a cache layer would shadow data-layer bugs rather than fix them.

WebSockets were considered for the carpool and buddy surfaces and rejected. Those surfaces are refresh-driven, with TanStack Query polling at 30 to 60 second intervals, and a websocket mini-service is a roadmap item only if real-time density demands it. A CMS, an email service, a payment provider, AI moderation, 3D or WebGL, and Storybook were each considered and rejected for v1 scope. Each is documented in the PRD as either an explicit non-goal or a roadmap item. The build does not carry the weight of features it does not yet need.

3.3 Dependency policy

Every dependency added during the build is justified against the ladder. No `any` types. No client-side AI SDK usage; the project rules forbid it. The `package-lock.json` is committed and `bun.lock` is removed so Vercel's resolution is deterministic and unambiguous.

4 Data model

The Prisma schema at `prisma/schema.prisma` is the single source of truth. The model has ten entities that carry user-generated content, plus the static station directory. Their relationships are shown in Figure 2.

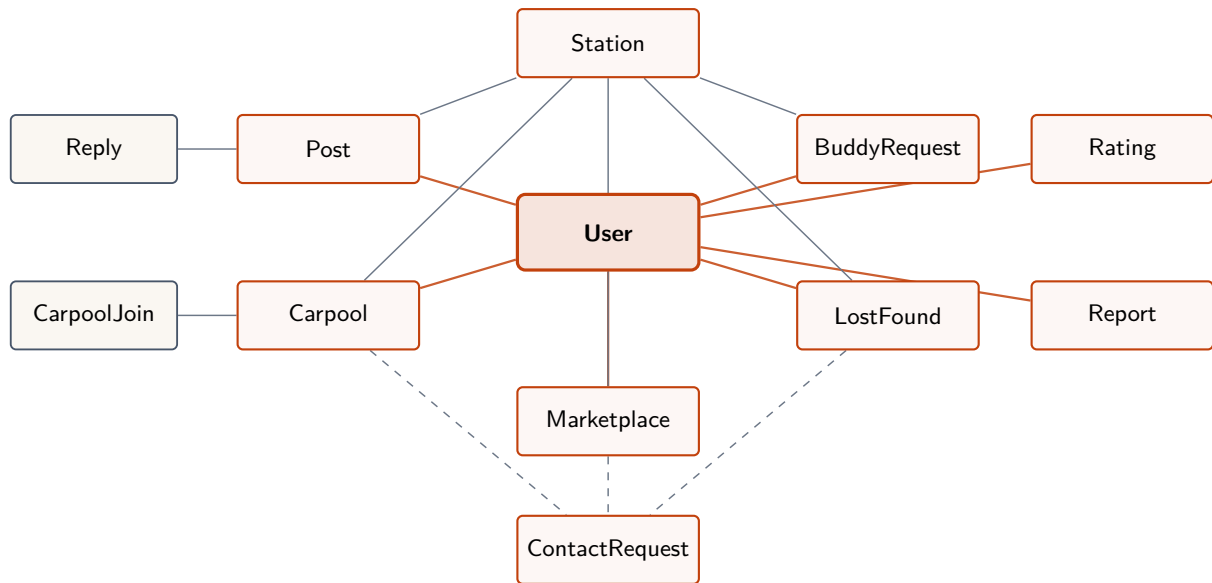


Figure 2: Entity-relationship diagram. Terracotta edges originate from **User**; slate edges from **Station** scope the user-generated content. Dashed edges show the polymorphic-ish contact-request association to **Carpool**, **LostFound**, and **Marketplace**. **Reply** and **CarpoolJoin** are owned children of **Post** and **Carpool** respectively. **ContactRequest** also carries `initiatorId` and `recipientId` foreign keys to **User** (omitted from the drawing for clarity).

User holds account credentials and the commuter profile: display name, home station, work or college station, usual travel window, bio, preferred language, an avatar (initials-based by default), and a cached `trustScore`. The `passwordHash` column is never selected in API responses.

Station is the static seed of Indian metro stations. Each row has a unique code (for example `RJ-AI` for Rajiv Chowk, `CS-CB` for Cubbon Park), a name, a city, and one or more line labels. The seed covers 39 stations across Delhi, Mumbai, Bengaluru, Hyderabad, Chennai, and Kolkata. Station rows are read-only from the application; they are populated by the seed script and not modified at runtime.

Post is the community feed. Posts can be scoped to a station or posted globally. They carry tags (help, info, alert, meetup, general), an optional image URL, and timestamps. **Reply** is one-to-many on **Post**. The original poster can mark a reply as the answer, in the style of a Q&A thread.

Carpool is the ride-share entity. It can be an offer or a request, with fields for origin station, destination area, departure time, seats, mode (auto, cab, personal vehicle), cost-split preference, and a women-only flag. **CarpoolJoin** is a separate entity representing a join request on a carpool; the poster accepts or declines. The lifecycle is `open` → `matched` → `completed`, or `open` → `expired`.

BuddyRequest is the travel-companion entity. It is distinct from **Carpool** because no vehicle is implied; it is purely about finding company for the metro ride itself, often for safety on an early-morning or late-evening leg. It supports a women-only filter. Lifecycle matches **Carpool**.

LostFound covers lost and found items. Two post types, lost something or found something, share the same table distinguished by a `type` field. Fields include item category, description, station, date, contact preference, and a status that progresses `active` → `reunited` → `closed`.

Marketplace is the local classifieds surface. Listings carry a title, description, price in INR, category, station catchment, condition, image URL, and a status of `available` → `reserved` → `sold`. No money flows through the platform; the listing expresses intent, and the transaction happens offline or via UPI directly between parties.

ContactRequest is the opt-in contact flow. It is polymorphic-ish: a single table with a `contextType` (carpool, lost-found, marketplace) and a `contextId`, plus `initiatorId` and `recipientId`. Contact information is not exposed until the recipient accepts. This is the trust primitive that the safety-sensitive features depend on.

Rating is the post-interaction rating. Both parties can rate each other 1 to 5 and leave a short note after a completed carpool, buddy match, or marketplace deal. Ratings feed into the trust score. **Report** is the moderation queue. Any user can flag any piece of user-generated content; the report row stores the reporter, the target, the reason, and a status. v1 has no automated action; reports are reviewed manually.

4.1 Trust score

Trust is computed in `lib/trust.ts` as a 0–100 score per user:

$$trustScore = \text{clamp}(b_{\text{profile}} + 4 \cdot \bar{r} + 0.2 \cdot n_{\text{completed}} - p_{\text{reports}}, 0, 100)$$

The score is recomputed on rating and report events and cached on the User row. It is intentionally simple and explainable; a learned model is a roadmap item, not a v1 feature. The constants are tuned by hand so that a new user with a complete profile and one five-star rating lands around 60, which is the threshold at which the verified-commuter badge appears.

4.2 Indices and query patterns

The schema defines indices for the actual query patterns: `Station(code)` unique, `Station(city, name)` composite for directory queries, `Post(stationId, createdAt)` for feed ordering, `Carpool(originStationId, status, departAt)` for active rides from a station, `LostFound` and `Marketplace` with `(stationId, category, status)` for filtering, and `Rating(ratedUserId)` for the trust-score aggregate. There are no joins that scan large tables; every list endpoint has a supporting index.

5 Authentication and session design

5.1 Provider and hashing

Authentication uses NextAuth.js v4 with the Credentials provider. Passwords are hashed with Node’s built-in `scrypt`, not `bcrypt`, because `bcrypt` requires native compilation and Vercel’s build environment does not reliably have the toolchain for it. `scrypt` is in `node:crypto`, ships with Node, and is more than adequate for a v1 password hashing workload. The hash format is `scrypt$<salt>$<hash>`, stored in `User.passwordHash`.

The registration flow (Figure 3) is: client posts to `/api/auth/register` with a Zod-validated payload; the handler checks email uniqueness, generates a salt, computes the scrypt hash, writes the User row, and returns a success envelope without ever selecting `passwordHash` in the response. The client then calls NextAuth’s `signIn` with the credentials provider, which invokes the `authorize` callback. The callback looks up the user by email, splits the stored hash, recomputes the scrypt hash of the submitted password with the stored salt, and compares in constant time. On success it returns the user object; on failure it returns null and NextAuth surfaces a generic invalid-credentials error.

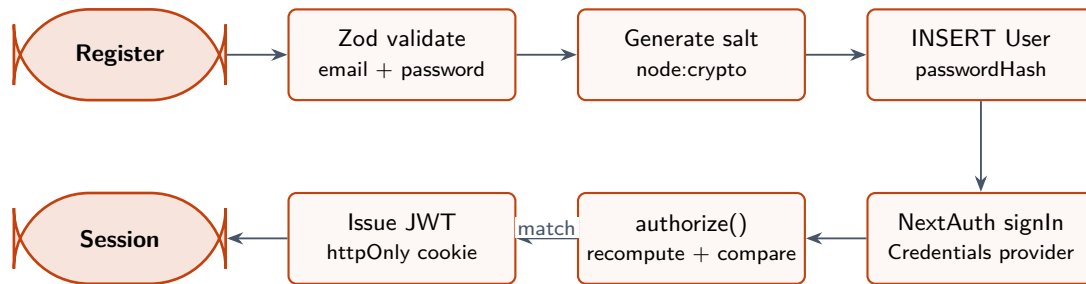


Figure 3: Authentication flow. Registration writes a bcrypt-hashed password; the subsequent NextAuth sign-in recomputes and compares in constant time, then issues a JWT in an httpOnly cookie.

5.2 Session shape

The session strategy is JWT, stored in an httpOnly secure cookie signed with `NEXTAUTH_SECRET`. The JWT callback enriches the token with `user.id`, `user.name`, `user.homeStationId`, and `user.trustScore`, so server components and API routes can read these without an extra database hit. The session callback exposes the same fields to the client.

5.3 Protected routes

The `(app)` route group requires a session; middleware redirects unauthenticated users to `/login` with a `callbackUrl`. API routes under `/api/`, except `/api/auth/` and the public read endpoints, require a valid session. There is no per-route role system in v1; all authenticated users have the same capabilities, and trust signals (the score, the badge) are advisory rather than gating.

5.4 Trust score as session data

Carrying the trust score in the session is a deliberate trade-off. It means the score can be stale by up to the JWT expiry window, but it avoids a database read on every authenticated request. For v1, with a 30-day session and a score that changes slowly, this is acceptable. If trust ever gates a write, for example requiring a minimum score to post a carpool, we will re-read the user row at the handler level rather than trust the JWT.

6 API design

6.1 Conventions

The API is REST-style route handlers under `/api/`. GET endpoints are public for read where the PRD allows (stations, public feed); POST and PATCH require a session. Every response follows the envelope `{ ok: boolean, data?: T, error?: string }`, which gives the client one shape to parse and one field to branch on. Zod schemas in `lib/validators.ts` are imported by both the client form and the server handler, so the validation contract is a single source of truth rather than a reimplemention.

The endpoint map covers auth, stations, feed (with replies and reports), carpools (with joins), buddies, lost-found (with contact), marketplace (with contact), ratings, contacts, reports, and user profiles. The full list is in the TechRD; the shape is consistent across all of them.

6.2 Validation contract

Every write endpoint follows the same five steps: parse the body with a Zod schema; return 400 with `ok:false` and the parse error on failure; authorise against the session; perform the database write inside a try/catch, returning 500 only for genuinely unexpected errors and never leaking the message; return 200 or 201 with `ok:true` and the data on success. The handler is thin; the schema is the contract.

Listing 1 shows the Zod schema for carpool creation, and Listing 2 shows the corresponding route handler. The schema is imported by both the client form (via a Zod resolver for react-hook-form) and the server handler, so the two cannot drift.

Listing 1: Zod schema for carpool creation (excerpt from `lib/validators.ts`).

```
import { z } from "zod";

export const createCarpoolSchema = z.object({
  originStationId: z.string().min(1),
  destinationArea: z.string().min(3).max(120),
  departAt: z.coerce.date(),
  seats: z.number().int().min(1).max(6),
  mode: z.enum(["auto", "cab", "personal"]),
  costSplit: z.enum(["split", "exact", "free"]).default("split"),
  womenOnly: z.boolean().default(false),
  type: z.enum(["offer", "request"]),
});

export type CreateCarpoolInput = z.infer<typeof createCarpoolSchema>;
```

Listing 2: Route handler for POST `/api/carpools` (excerpt).

```
import { NextResponse } from "next/server";
import { getServerSession } from "next-auth";
import { authOptions } from "@/lib/auth";
import { prisma } from "@/lib/db";
import { createCarpoolSchema } from "@/lib/validators";

export async function POST(req: Request) {
  const session = await getServerSession(authOptions);
  if (!session?.user?.id) {
    return NextResponse.json(
      { ok: false, error: "Unauthorized" },
      { status: 401 }
    );
  }
  const parsed = createCarpoolSchema.safeParse(await req.json());
  if (!parsed.success) {
    return NextResponse.json(
      { ok: false, error: parsed.error.message },
      { status: 400 }
    );
  }
  try {
    const carpool = await prisma.carpool.create({
      data: { ...parsed.data, userId: session.user.id },
    });
    return NextResponse.json(
      { ok: true, data: carpool },
      { status: 201 }
    );
  } catch (err) {
    console.error("createCarpool failed", err);
  }
}
```

```

return NextResponse.json(
  { ok: false, error: "Internal error" },
  { status: 500 }
);
}
}

```

6.3 Request flow

Figure 4 traces a representative write: Devika, the primary persona, posts a carpool request. The browser POSTs to `/api/carpools`; the route handler hands the body to the Zod schema, which either returns a parsed value or surfaces a 400. The handler then calls `prisma.carpool.create`, which issues an INSERT against SQLite. On success the handler returns 201 with the standard envelope; the client's TanStack Query mutation invalidates the relevant query keys and the list refetches.

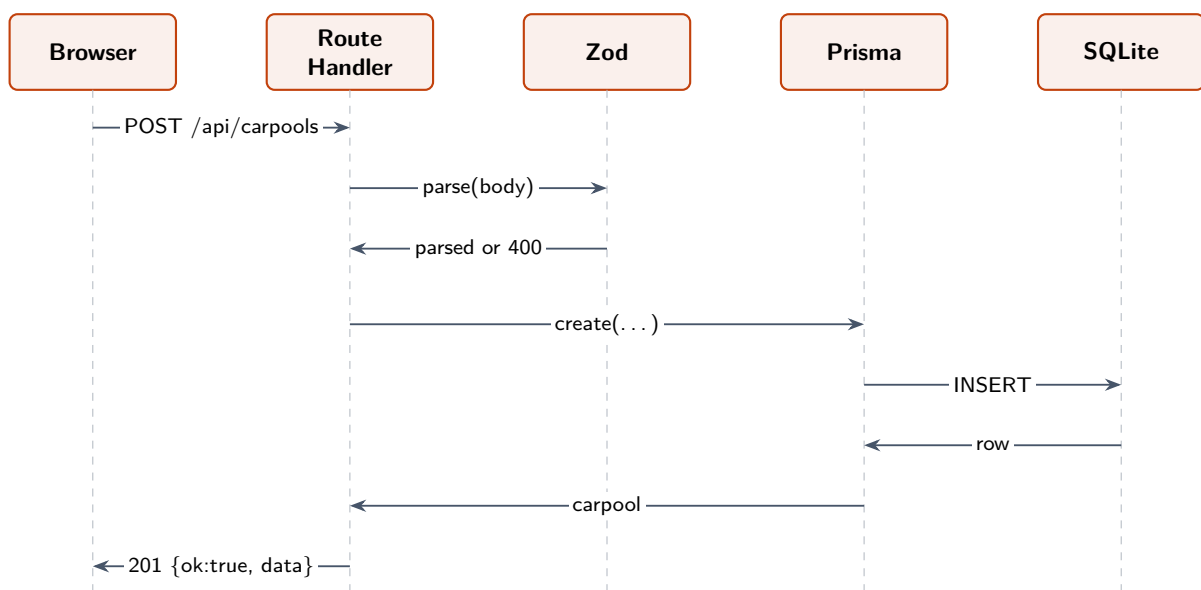


Figure 4: Request flow for a representative write: Devika posts a carpool request. The handler validates with Zod, persists via Prisma, and returns the standard envelope. Return arrows (right to left) are responses.

6.4 Errors and the envelope

The `ok` flag is the contract. A 200 with `ok:false` is impossible by construction: parse failures return 400, authorisation failures return 401, and unexpected failures return 500. The client therefore checks `response.ok` first, then `payload.ok`, and only then reads `payload.data`. This is verbose in the calling code but unambiguous, and the TanStack Query wrappers hide most of it.

7 Frontend architecture

7.1 Route groups

The App Router uses three route groups, which do not affect the URL: (`public`) for the landing page, station directory, public station pages, and about; (`auth`) for login and register with minimal chrome; and (`app`) for the authenticated dashboard and feature surfaces. Public routes

are server-rendered for crawlability and carry per-route metadata. Auth routes are a server-rendered shell with a client form, so the auth logic never ships to the browser. App routes are server-rendered lists with client interactivity layered on top.

7.2 Rendering strategy

The split is deliberate. The landing page, the station directory, and public station pages must be crawlable and fast on a slow mobile connection, so they are pure server components with metadata. The dashboard and the feature surfaces (feed, carpools, buddies, lost-found, marketplace) are server-rendered for the first paint, then hydrate TanStack Query for subsequent interactions. This balances crawlability of the public parts with the interactivity the application surfaces need.

7.3 State management

Server state is TanStack Query, with `staleTime` values tuned per surface: 60 seconds for feeds, 30 seconds for carpool and buddy lists. Client UI state is Zustand, used sparingly for cross-component concerns like the SOS dialog open flag and the active station filter. Form state is react-hook-form with a Zod resolver; forms call API routes via TanStack Query mutations, and successful mutations invalidate the relevant query keys.

7.4 Component contract

`shadcn/ui` primitives are the only button, card, dialog, input, and so on used in the application. There are no custom re-implementations. Feature components such as `CarpoolCard`, `FeedItem`, and `StationHeader` live under `components/<feature>` and accept typed props. Every data-driven component has empty, loading, and error states; this is mandatory, not optional, and is checked in review. Every interactive element is keyboard-operable with a visible focus ring.

7.5 Motion level

Motion is Level 1: refined interface motion only. Hover, focus, press, dialog open and close, tab switch, section entry. There is a small amount of directional motion, an x-axis slide for arriving content, that echoes the metro motif without becoming a gimmick. `prefers-reduced-motion` is respected; all motion degrades to opacity-only or none.

8 Deployment and CI/CD

8.1 Package manager and lockfile

npm is the package manager of record. `package-lock.json` is committed; `bun.lock` was removed. The reason is operational: Vercel's build resolution is deterministic with npm and a committed lockfile, and bun's lockfile resolution has been observed to be ambiguous in some Vercel configurations. The build script is plain `next build`, with no standalone output and no custom copy step, so Vercel's default Next.js builder works without a `vercel.json` override.

8.2 Topology

Figure 5 shows the deployment topology. A push or pull request to `main` on GitHub triggers the GitHub Actions workflow. The workflow runs the CI gate; on push to `main` only, it additionally deploys to Vercel. Vercel builds the Next.js application and serves it from its edge network.

The database in production is, for v1, an ephemeral SQLite file under `/tmp`, with a documented migration path to Turso or Postgres.

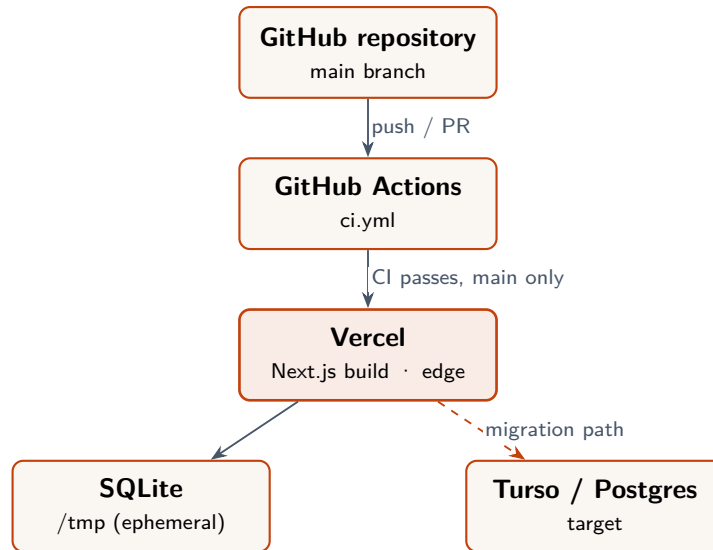


Figure 5: Deployment topology. Pushes to `main` trigger the GitHub Actions pipeline, which on a green build deploys to Vercel. Production data is ephemeral on Vercel’s read-only filesystem; the dashed edge marks the documented migration to Turso or Postgres.

8.3 CI/CD pipeline

A single GitHub Actions workflow at `.github/workflows/ci.yml` runs on every push and pull request to `main`. The steps are shown in Figure 6: `checkout`, `set up Node 20`, `npm ci`, `npm run lint`, `npx tsc --noEmit`, `npx prisma generate`, `npm run build`. On push to `main` only, an additional job deploys to Vercel via the Vercel CLI using `VERCEL_TOKEN`, `VERCEL_ORG_ID`, and `VERCEL_PROJECT_ID` secrets, targeting production.

We deliberately avoided a separate `deploy.yml`. The Ponytail principle is to not scaffold for a speculative future; one workflow that covers lint, type-check, build, and deploy keeps the pipeline auditable in a single file, and a future split, if it ever becomes necessary, is a small refactor.

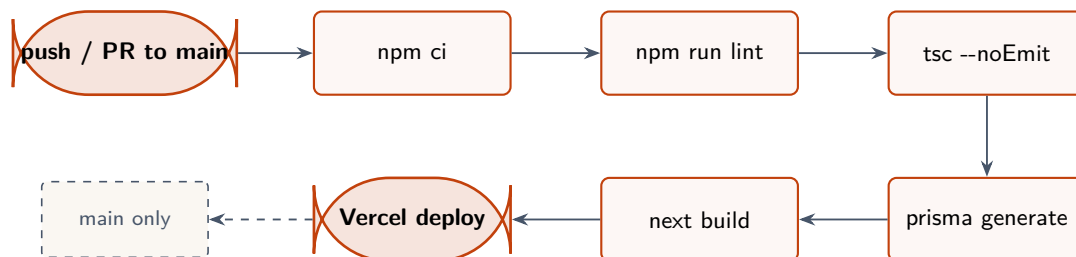


Figure 6: CI/CD pipeline. A single workflow runs lint, type-check, Prisma generation, and build on every push and PR. The deploy step runs only on `main`.

8.4 The database limitation

Vercel’s filesystem is read-only except for `/tmp`. SQLite works locally and during a single warm Vercel instance, but data does not persist across cold starts in production. We document this honestly rather than pretend otherwise: v1 on Vercel has an ephemeral database. The seed script runs on cold start, so the application always has its 39 stations and three demo users, but user-generated content does not survive a redeploy.

The migration path is documented and intentionally not auto-applied. Switching to Turso (libSQL) or Postgres requires changing the Prisma datasource provider and the `DATABASE_URL` environment variable. No application code changes, no schema changes. The schema uses only portable types; there are no SQLite-specific JSON-array primitives that would not migrate. Section 11 covers the roadmap.

9 Security and privacy

Passwords are hashed with scrypt, not stored in cleartext, and never selected in API responses. Sessions are JWTs in httpOnly secure cookies, signed with `NEXTAUTH_SECRET`, with NextAuth's built-in CSRF protection enabled. Every write endpoint parses its body with Zod; no route handler reads `req.body` directly. A serialiser helper strips sensitive fields from User and other entities before they leave the process.

Contact information is never exposed until both parties accept a `ContactRequest`. The trust primitives (ratings, reports, the trust score) are advisory in v1 but visible, so a user can make an informed decision about whether to share a ride or meet at a station. The SOS shortcut on every authenticated screen surfaces the user's last station and a prompt to share live location or call 112; on the web this is informational and uses device capabilities where available.

Reports are stored with the reporter ID for audit. There is no automated action in v1; reports go into a manual queue. Rate limiting is not implemented in v1 and is documented as a limitation. No third-party analytics or tracking is shipped in v1. The `.env` file, the SQLite database file, `node_modules`, and the `.next` build output are gitignored and never committed.

10 Accessibility and performance

The accessibility contract is non-negotiable. Semantic landmarks (`header`, `main`, `nav`, `footer`) on every page; logical heading order preserved per route; keyboard-operable controls with visible focus rings; touch targets at least 44 pixels; AA contrast on all text; `prefers-reduced-motion` respected; no colour-only meaning, so line colours are always paired with a label. These are checked in review, not bolted on at the end.

Performance targets for v1: largest contentful paint under 2.5 seconds on a typical 4G mobile profile, cumulative layout shift under 0.1, interaction to next paint under 200 milliseconds. Public pages are server-rendered for fast first paint and crawlability. Image and font budgets are kept proportionate; there is no decorative WebGL in v1, and the metro-line motif is delivered with CSS and SVG rather than canvas. The application is verified at 390, 768, and 1440 pixels, and at intermediate widths where wrapping can fail.

11 Limitations and future work

v1 has known limits. We list them honestly.

The Vercel filesystem limitation makes SQLite ephemeral in production. The migration to Turso or Postgres is documented and small (a datasource change and an environment variable), but until it is applied, user-generated content does not survive a redeploy. This is the single most important limitation to fix before any real user is onboarded.

There is no rate limiting on the API. A determined abuser could spam registrations or feed posts; NextAuth's built-in CSRF protection mitigates the worst classes of attack, but a proper rate-limit layer is a v1.1 item. Content moderation is manual: the report queue exists, but no

automated classification or filtering is applied to user-generated content. AI-assisted moderation is a roadmap item, to be considered only when there is enough traffic to train and evaluate a model honestly.

There are no native mobile apps. The web app is responsive and PWA-ready in metadata, but the delivery target for v1 is a web deployment. A PWA install flow and offline-first behaviour are roadmap items. Real-time train and crowd data is out of scope; v1 is community-led, and metro corporation partnerships for live data are a future cycle. There is no in-app payment; marketplace listings express intent, and transactions happen offline or via UPI directly between parties. In-app UPI escrow is a roadmap item.

Localisation is English-only in v1. The schema and UI strings are structured so Hindi and one regional language per launch city can be added without schema changes. The trust score is a hand-tuned linear function; a learned model is a roadmap item, to be considered only when there is enough data to train one honestly.

12 Conclusion

MetroMitra v1.0 is a deliberate, minimal build that tests a specific product thesis: that the metro station is a viable primary unit of community for Indian commuters. The architecture reflects that minimality. One Next.js process, one database through Prisma, one auth provider, one CI pipeline, one deploy target. Every additional piece of infrastructure was considered against the Ponytail ladder and, for v1, rejected. This document records not only what we built but what we chose not to build, and why.

The known limits (ephemeral SQLite on Vercel, manual moderation, no rate limiting, English-only) are documented here and in the README. The migration paths are documented too. What v1 delivers is a deployed, usable, accessible web application that lets a real Indian metro commuter register, anchor to a station, post a carpool, find a buddy, list a lost item, browse a local marketplace, and rate a fellow commuter. That is enough to test the thesis. Everything beyond it is roadmap.

References

1. MetroMitra Product Requirements Document, [docs/PRD.md](#).
2. MetroMitra Technical Requirements Document, [docs/TechRD.md](#).
3. Ponytail SKILL: simplest correct implementation ladder, [docs/reference/Ponytail-SKILL.md](#).
4. Master Playbook, [docs/reference/Master-Playbook.md](#).
5. Anti-patterns reference, [docs/reference/anti-patterns.md](#).
6. Next.js App Router documentation, <https://nextjs.org/docs/app>.
7. Prisma documentation, <https://www.prisma.io/docs>.
8. NextAuth.js v4 documentation, <https://next-auth.js.org>.
9. Node.js `crypto.scrypt` documentation, <https://nodejs.org/api/crypto.html#cryptoscript>.
10. Vercel deployment documentation, <https://vercel.com/docs>.
11. Turso (libSQL) documentation, <https://docs.turso.tech>.
12. TanStack Query documentation, <https://tanstack.com/query/latest>.